

CYBERSECURITY INTERNSHIP

Organization name: Elevate labs

Student Name: Malladi Siva Shankar

University: Aditya University

Domain: Cybersecurity

Duration: 1st January 2026 to 30th April 2026

Task 6: Introduction to Cryptography

1. Symmetric vs Asymmetric encryption :-

Encryption:-

Encryption is the process of converting readable data (plaintext) into an unreadable, coded format (ciphertext) using algorithms and a digital key, preventing unauthorized access and ensuring confidentiality, making data useless to anyone without the correct decryption key.

It's a fundamental tool in cybersecurity, scrambling sensitive information like passwords, financial details, and messages, so even if intercepted, it remains secure.

Symmertic Encryption :-

- Symmetric key encryption, also known as symmetric cryptography, is a type of cryptography that uses the same key to encrypt and decrypt a message. This means that both the sender and the receiver of a message must have access to the same secret key to encode and decode it.
- It uses one key for both encryption and decryption.
- Faster and more efficient for large amounts of data.
- Requires a secure method to share the key between sender and receiver.
- Common algorithms include AES, DES, Blowfish.

Asymmetric Encryption :-

- Asymmetric key encryption is one of the most common cryptographic methods that involve using a single key and its pendent, where one key is used to encrypt data and the second one is used to decrypt an encrypted text. The second key is kept highly secret, while the first one which is called a public key can be freely distributed among the service's users.
- It uses two keys a public key for encryption and a private key for decryption.
- More secure but slower than symmetric encryption.
- No need to share the private key, reducing the risk of exposure.
- Common algorithms include RSA, ECC, Diffie-Hellman.

2. Encrypt files using AES

AES (Advanced Encryption Standard) is a symmetric key encryption algorithm, meaning the same secret key is used to encrypt and decrypt data. It is widely used to protect files, disks, and network communications because it is fast, secure, and standardized

How AES encryption works (conceptually)

- Plaintext (original file data) is taken as input.
- A secret key (128, 192, or 256 bits) is generated or derived from a password.
- AES performs multiple rounds of substitution, permutation, and mixing on the data.
- The result is ciphertext, which looks like random data.
- The same key is required to decrypt the ciphertext back to plaintext.

Why AES is secure

- Approved by NIST
- No practical attacks on full AES
- Resistant to brute force due to large key space
- Efficient in both hardware and software

AES is a symmetric block cipher that encrypts data using the same secret key, providing fast and strong security through multiple encryption rounds.

Step 1 :- Check OpenSSL version openssl
version

```
(kali@kali)-[~]  
$ openssl version  
OpenSSL 3.5.2 5 Aug 2025 (Library: OpenSSL 3.5.2 5 Aug 2025)
```

Step 2 :- Create a Sample File create
a sample file of the choice

```
(kali@kali)-[~]  
$ echo "This is a confidential file" > file.txt
```

Step 3 :- Encrypt File Using AES-256

```
(kali@kali)-[~]  
$ openssl enc -aes-256-cbc -salt -in file.txt -out file.enc  
enter AES-256-CBC encryption password:  
Verifying - enter AES-256-CBC encryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.
```

Step 4 :- Verify Encrypted File

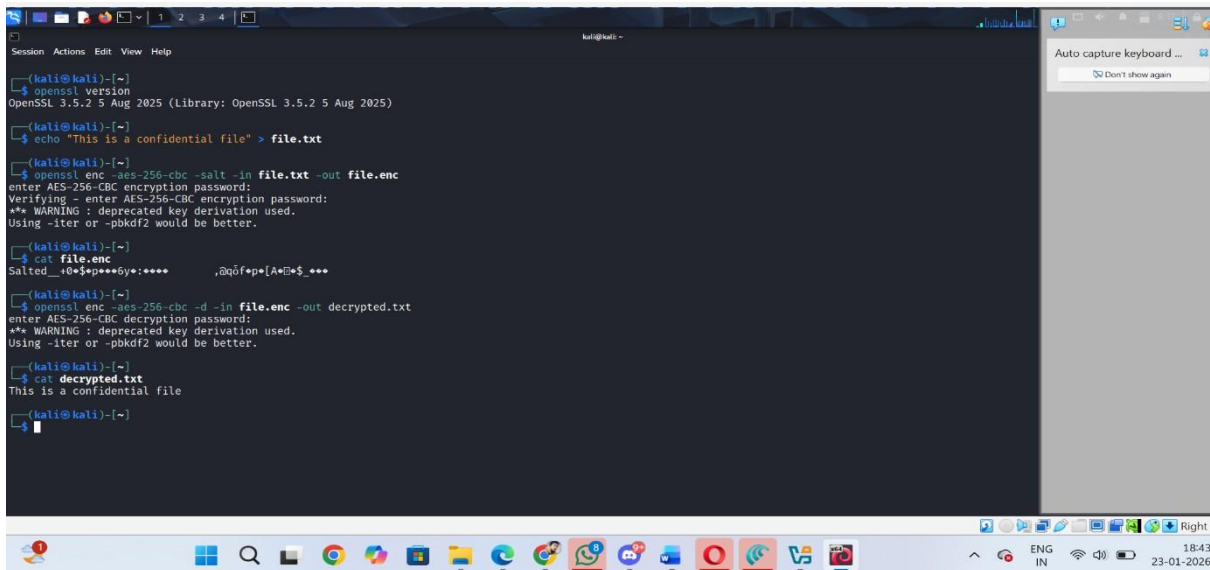
```
(kali@kali)-[~]  
$ cat file.enc  
Salted__+0+$p+++6y+:++++,dqõf♦p♦[A♦♦$$_+++
```

Step 5 :- Decrypt the AES Encrypted File

```
(kali@kali)-[~]  
$ openssl enc -aes-256-cbc -d -in file.enc -out decrypted.txt  
enter AES-256-CBC decryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.
```

Step 6 :- Confirm Decryption

```
(kali@kali)-[~]  
$ cat decrypted.txt  
This is a confidential file
```



```
(kali@kali)-[~]
└─$ openssl version
OpenSSL 3.5.2 5 Aug 2025 (Library: OpenSSL 3.5.2 5 Aug 2025)

(kali@kali)-[~]
└─$ echo "This is a confidential file" > file.txt

(kali@kali)-[~]
└─$ openssl enc -aes-256-cbc -salt -in file.txt -out file.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(kali@kali)-[~]
└─$ cat file.enc
Salted__0x$pxx0y0:0000      ,@Q0f*p*[A#E*$_000

(kali@kali)-[~]
└─$ openssl enc -aes-256-cbc -d -in file.enc -out decrypted.txt
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(kali@kali)-[~]
└─$ cat decrypted.txt
This is a confidential file

(kali@kali)-[~]
└─$
```

3. Generate RSA keys :-

- RSA (Rivest–Shamir–Adleman) is an asymmetric cryptographic algorithm that uses two different keys: a public key and a private key.

How RSA key generation works (conceptual)

Two large prime numbers are generated.

The primes are multiplied to form a modulus (n).

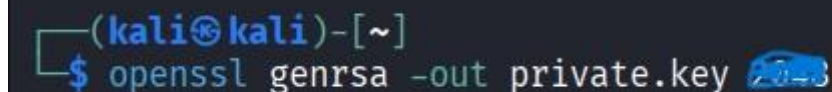
A public exponent (e) is selected.

A private exponent (d) is mathematically calculated.

The result is a key pair: public key (e, n) and private key (d, n).

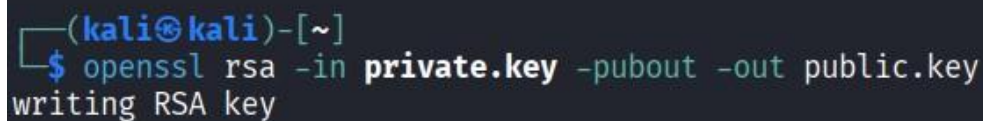
Generate RSA keys (OpenSSL)

Step 1 :- Generate a 2048-bit RSA private key



```
(kali@kali)-[~]
└─$ openssl genrsa -out private.key 2048
```

Step 2 :- Generate the public key from the private key



```
(kali@kali)-[~]
└─$ openssl rsa -in private.key -pubout -out public.key
writing RSA key

(kali@kali)-[~]
└─$
```

Step 3 :- Verify the keys

```
(kali@kali)-[~]
$ openssl rsa -in private.key -check
RSA key ok
writing RSA key
-----BEGIN PRIVATE KEY-----
MIIEVQIBADANBgkqhkiG9w0BAQEFAASCBCwggSjAgEAAoIBAQC2c0wd3EMuedZW
4R9jKqNU3+coJkJDV0S05xCoHhXkckqMpyQ+Zv2utDsyuygE7CgFugNXH20w6vNg
Bfwqy12y0UFIInKCjn7Bw+RF5YJ8ZgeHnOs72DGjvZIB0LCQhFPyQES6duI1w3zh4
KzYY6e7n8AoygV50tDhKWuWlx+MGdBQIHr+5/YPHUIw4uYtoI5/OasSJ00qW/3zt
2m+RWquaVfk1lZ5XckzarBfZzcuC4zxP+UP96iP5lnaYiNPDIN6BmD3K/p7aXBTh
b471rvwhtJ5H6dqzUhrz3jJE9C/5ni3P2RoFgd5QuX/xfv17G//aTdYmMQOCc6tZ
crbtbWV9AgMBAECggEAAp0TJDUBU9YsRDRzdFzbOhnq4jr7zFGW8Fmg83h9oAq
1XDeALWfApDS1bdVty0czGaU5USabI6HvtuMp08dwcD7gXkdYuJHGN2b0brYMDgW
N5DS6HqWkPfnX3xrZyH94KP1l81iEg/+xNWEtGumRTxAoErUgMKkLlJmQyWYBJIs
n+gTmcMeE57VUzjaUGx/+wGnlWYDHyUinvdwQ2LFb20cCGhV96/wkXm4A5VNUf+A
idrjoFQzsMikLuyXpMS/IzdbI3OysFST5U6xFeGb9yaJPNIHhChnMkBWjzG7pjIA
RH8DsLO8MFuBQ9811zCJEoEKA2uCT3L/Ohn+OT8KwKBgQD+q2yeQBjNk4aIQj9e
Tzrvb5cLQ840rZ5Unbj2Q8NQ9++aULkeV6PXCS5amOAt3Xa8kkD+M10LuOFZ6Uvu
Zg3s65TB5G2NA1zsuG0RCzo705iWbYF5xgemnX17gb0E3vFJLb2unrIEQjomFYt
iBqnkwV2iUw8jrQ0FFVvKCB1wKBgQC3Z0rjqkEbLSZCyZRkQ8szNqVyla9abiH0
+2ECJGbsF+DQI4g98cVMWvwyZvs9HUwaaP3IelaigA28wsJjoIh/StSaPuz1RVJ6
EY9aGvphC3N8rEqrZs7YCbzCLJPGLkcNp+MygkLGBTO1uLYiungQ0S1xg9w34yrR
v3Yrzf8QyWKB8QCJEep1LZ90CjELEd8Y/+x8cZb6P1LNS5nknjTVYkYukwKVMXj6
x+D9cECA/AAW/gXoCLGZMm4dOdZ5QBTBY5EpEVLnr4bB0tRuMkylrD+165vxPd64
hSc19RN4JENOWh3u+F3K8IwVx/49onCni9euxG1bIoWQxOcKkIhzm9xcwKBgE04
lpWpN+SDbBqu2WymDUaw+ZR3Ze/Dc1S9CzRYN1q6ZxP8Af9i7tmpbYqjRaGD1mlA
T6q8E8wz8pTJFPuxgusLmJSSg0Wfe/nNBjJ6aQtb2GpaHplans36dl+5qf56be
S5EWu5q40f7vmpCht4eN2ray+GQnp/3LU8d54q1lAoGAK/w0jQdQzptPmslQw9GJ
V6hVSYc57A7WlpHn9FFFoIKDvN6mdxd+SdYncd6eJoHbFXjMwtng05qLH0d1TiXf
oRmIPkMmlJEEcU50FoEI3pLPKJvwm68bq3f05fq0bzp7CUIl048nIIYbzbyKztXl
PMGbzG10KZaFnlQ/1qZ4xc=
-----END PRIVATE KEY-----
```

```
(kali@kali)-[~]
$ openssl rsa -in public.key -pubin -text -noout
Public-Key: (2048 bit)
Modulus:
00:b6:73:4c:1d:dc:43:2e:79:d6:56:e1:1f:63:2a:
a3:54:df:e7:28:26:42:43:57:44:b9:d3:10:a8:1e:
1c:64:72:4a:8c:a7:24:3e:ce:fd:ae:b4:3b:32:bb:
28:04:ec:28:05:52:09:d7:1f:6d:30:ea:f3:60:05:
fc:2a:cb:5d:b2:d1:41:48:9c:a0:a3:9f:b0:70:f9:
11:79:60:9f:19:81:e1:e7:3a:ce:f6:0c:68:ef:64:
86:ce:2d:c4:21:14:fc:90:11:2e:9d:b8:89:70:df:
38:78:2b:36:18:e9:ee:e7:f0:0a:32:81:54:8e:b4:
31:ca:5a:e5:a5:c7:e3:06:74:14:08:1e:bf:b9:fd:
83:c7:50:8c:38:b9:8b:68:23:9f:ce:6a:c4:89:d3:
4a:9e:ff:7c:ed:da:ef:91:da:ab:2a:5f:f9:35:95:
9e:57:0a:4c:da:a6:17:d9:cd:cb:82:e3:3c:4f:f9:
43:fd:ea:23:f9:96:76:98:88:d3:c3:88:de:81:98:
3d:ca:fe:9e:da:5c:14:e1:6f:8e:f5:ae:fc:21:b4:
9e:47:e9:da:b3:52:1a:f3:de:32:44:f4:2f:f9:9e:
2d:cf:d9:1a:05:81:de:50:b9:7f:f1:7e:fd:7b:1b:
ff:da:4d:d6:26:31:03:82:73:ab:59:72:b6:ed:6d:
65:7d
Exponent: 65537 (0x10001)
```

4. Digital signatures

- A digital signature is a cryptographic code that authenticates digital documents, verifying the sender's identity and ensuring the content hasn't been altered, acting as a secure, electronic equivalent to a handwritten signature by using a pair of private and public keys to create a unique, unforgeable digital fingerprint for the data
- It confirms authenticity, integrity, and origin, vital for secure online transactions, contracts, and government filings, by binding the signature to the document's content through Public Key Infrastructure(PKI) and Certified Authorities.

How it works

Key Generation: A unique pair of keys is created: a secret private key for signing and a public key for verification.

Hashing: A mathematical function creates a fixed-length "hash" (digital fingerprint) of the document.

The sender encrypts the hash using their private key, creating the digital signature.

Verification: The recipient uses the sender's public key (often from a Digital Signature Certificate) to decrypt the hash. If the decrypted hash matches a newly generated hash of the document, the signature is valid, proving the sender's identity and that the document is unchanged.

5. Hash files and verify integrity

Hashing :-

- Hashing is the process of converting file data into a fixed-length hash value (digest) using a hash algorithm.
- Even a tiny change in the file produces a completely different hash.
- Hashing is one-way (cannot be reversed). **Integrity**

verification :-

- Integrity verification means checking whether a file has been altered or tampered with by comparing its hash value
- Same hash → file is unchanged
- Different hash → file is modified or corrupted

Common hash algorithms

MD5 (fast but broken)

SHA-1 (deprecated)

SHA-512 (stronger)

Step 1:- Generate SHA-256 hash

```
(kali@kali)-[~]  
$ openssl sha256 file.txt  
SHA2-256(file.txt)= c60228f0dc9b7e20813c47ec5dc9177d0b324efa2d191318440ac1c3771788
```

Step 2 :- Generate MD5 has

```
(kali@kali)-[~]  
$ openssl md5 file.txt  
MD5(file.txt)= 3dd961eb4e899ee9df5e01a021b7a5dd
```

Integrity Step 1 :- Generate original hash

```
(kali@kali)-[~]  
$ openssl sha256 file.txt > hash.txt
```

Step 2 :- Later, re-hash the file

```
(kali@kali)-[~]  
$ openssl sha256 file.txt  
SHA2-256(file.txt)= c60228f0dc9b7e20813c47ec5dc9177d0b324efa2d191318440ac1c3771788
```

Step 3 :- Compare hashes

The hashes are been matched

6. Encryption algorithms :-

An encryption algorithm is a set of mathematical rules and steps used to scramble readable data (plaintext) into unreadable code (ciphertext) and back again, using a secret key, to secure information like messages, files, and transactions from unauthorized access

1. Symmetric Encryption Algorithms :- AES(Advance Encryption Standard)

Advanced Encryption Standard (AES) is a symmetric block cipher adopted by the U.S. government to secure sensitive data in both hardware and software. It uses fixed 128-bit block sizes with key lengths of 128, 192, or 256 bits. Unlike DES, AES is based on a substitution-permutation network (SPN) rather than a Feistel structure, making it faster and more secure.

Triple DES

Triple DES (3DES) is an enhanced version of DES, applying the algorithm three times with three keys for a total length of 168 bits, though its effective security is about 112 bits. Stronger than DES but slower in software, it is better suited for hardware. Today, it has been largely replaced by the faster and more secure AES.

Twofish

Twofish algorithm is successor of blowfish algorithm. It was designed by Bruce Schneier, John Kelsey, Dough Whiting, David Wagner, Chris Hall and Niels Ferguson. It uses block ciphering It uses a single key of length 256 bits and is said to be efficient both for software that runs in smaller processors such as those in smart cards and for embedding in

hardware .It allows implementers to trade off encryption speed, key setup time, and code size to balance performance.

2. Asymmetric Encryption

Asymmetric encryption (public-key encryption) uses a mathematically linked key pair, a public key for encryption and a private key for decryption. The private key cannot be derived from the public key, making it highly secure. Unlike symmetric encryption, it enables safe key exchange since the private key is never shared.

1. RSA (Rivest, Shamir and Adleman)

RSA is an asymmetric encryption algorithm named after its creators. It is based on the mathematical difficulty of factoring large composite numbers. The algorithm uses a key pair: a public key for encrypting plaintext and a private key for decrypting ciphertext. By keeping the private key secret, RSA ensures secure communication and remains one of the most trusted methods for data protection.

2. Elliptic Curve Cryptography (ECC)

Elliptic Curve Cryptography (ECC) is an asymmetric encryption technique that relies on the mathematics of elliptic curves to provide strong security with much smaller key sizes compared to traditional algorithms like RSA. This efficiency makes ECC faster and more suitable for devices with limited processing power, such as smartphones and IoT systems. It is widely used in secure communications, digital certificates, and cryptocurrencies, where both performance and security are critical.

3. Digital Signature Algorithm (DSA)

Digital Signature Algorithm (DSA) is an asymmetric algorithm designed specifically for creating and verifying digital signatures rather than encrypting data. It ensures authenticity by confirming the identity of the sender and integrity by verifying that the data has not been altered during transmission. DSA provides non-repudiation, meaning the sender cannot deny their involvement once a signature is generated. It is commonly used in authentication processes, software distribution, and digital certificates.

7. Real-world usage (HTTPS, VPN)

HTTPS

HTTPS (HyperText Transfer Protocol Secure) protects data exchanged between a browser and a web server.

How cryptography is used in HTTPS:

- The server presents an SSL/TLS certificate (uses RSA or ECC).
- The browser verifies the certificate using a trusted Certificate Authority (CA).
- A secure key exchange occurs (RSA / ECDHE).
- A symmetric session key is generated.
- Actual data is encrypted using AES or ChaCha20.

Example :-

Example: <https://gmail.com> How crypto is used:

- **RSA / ECC** → authenticates the website & exchanges keys
- **AES / ChaCha20** → encrypts web data
- **SHA-256** → ensures data integrity

```
(kali@kali)-[~]
$ openssl s_client -connect google.com:443
Connecting to 142.251.43.46
CONNECTED(00000003)
depth=2 C=US, O=Google Trust Services LLC, CN=GTS Root R4
verify return:1
depth=1 C=US, O=Google Trust Services, CN=WE2
verify return:1
depth=0 CN=*.google.com
verify return:1
Certificate chain
 0 s:CN=*.google.com
  i:C=US, O=Google Trust Services, CN=WE2
  a:PKEY: EC, (prime256v1); sigalg: ecdsa-with-SHA256
  v:NotBefore: Dec 29 19:51:10 2025 GMT; NotAfter: Mar 23 19:51:09 2026 GMT
 1 s:C=US, O=Google Trust Services, CN=WE2
  i:C=US, O=Google Trust Services LLC, CN=GTS Root R4
  a:PKEY: EC, (prime256v1); sigalg: ecdsa-with-SHA384
  v:NotBefore: Dec 13 09:00:00 2023 GMT; NotAfter: Feb 20 14:00:00 2029 GMT
 2 s:C=US, O=Google Trust Services LLC, CN=GTS Root R4
  i:C=BE, O=GlobalSign nv-sa, OU=Root CA, CN=GlobalSign Root CA
  a:PKEY: EC, (secp384r1); sigalg: sha256WithRSAEncryption
  v:NotBefore: Nov 15 03:43:21 2023 GMT; NotAfter: Jan 28 00:00:42 2028 GMT
Server certificate
-----BEGIN CERTIFICATE-----
MIINH2CCDS2gAWIBAgIRAIgZNNiwWk63EnFupzWL8BMwCgYIKoZizj0EAWIwOzEL
MAKGA1UEBHMCMVVMxHjAcBgNVBAoTFUdvb2dsZSBUCnVzdCBTZj0EAWIwOzEL
A1UEAxMDV0UyYMB4XDTI1MTIyOTUyOTUyOTUyOTUyOTUyOTUyOTUyOTUyOTUy
A1UEAwMKi5nb29nbGUuY29tMFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAErADW
V/ZutGyUcTixvqr716bebqwe2tNwjLpCLZrbRTcUbwWTXaFng+GFCEK0GJEbdX5J
-----END CERTIFICATE-----
No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: ecdsa_secp256r1_sha256
Negotiated TLS1.3 group: X25519MLKEM768

SSL handshake has read 6438 bytes and written 1759 bytes
Verification: OK

New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
Server public key is 256 bit
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
```

VPN

A VPN creates a secure encrypted tunnel between a user and a remote network over the internet.

How cryptography is used in VPNs:

- User authenticates with the VPN server.
- Secure key exchange takes place.

- An encrypted tunnel is established.
- All traffic passes through this tunnel securely.

Example :-

OpenVPN, WireGuard, NordVPN How crypto is used:

- AES-256 → encrypts all traffic
- RSA / ECDH → secure key exchange

8. Document Findings

SUMMARY :-

Cryptography is used to secure data through encryption and hashing techniques. Symmetric encryption uses a single secret key for both encryption and decryption and is fast, making it suitable for data encryption (e.g., AES). Asymmetric encryption uses a public–private key pair and is mainly used for secure key exchange and authentication (e.g., RSA).

AES is a symmetric algorithm that encrypts files by converting plaintext into ciphertext, ensuring confidentiality. RSA generates a public key for encryption and a private key for decryption and digital signatures. Digital signatures provide authentication, integrity, and non-repudiation by signing data with a private key and verifying it with a public key.

Hashing generates a fixed-length hash value to verify data integrity, where any change in the file alters the hash. Symmetric algorithms are fast, while asymmetric algorithms solve key distribution problems.

Modern systems combine both methods. Technologies like HTTPS and VPNs use these cryptographic techniques to secure communication and network traffic.